Frontend overview and general plan

Documents, research & tasks done to take into account:

- Vision document
- European Shelter Dog Adoption Platform.pdf
- Trello Kanban tasks “Website design vol 1”, “Frontend development vol 1”, “Figure out the correct framework to use”
- Various research by team members that has been communicated in the Slack channel

Framework

Next.js (React-based for SSR and dynamic routing). The project needs to load quickly, rank well on Google, support multiple languages, and display dynamic dog/shelter profiles efficiently.

- SEO Optimization (Google can index dog profiles for better visibility)
- Fast page loads with SSR (Essential for performance)
- Dynamic routing for dogs & shelters
- Built-in API routes (No need for a separate backend for simple tasks)
- Image Optimization (Loads dog photos efficiently)
- Internationalization support (Important for an EU-wide platform)
- Support for multiple languages
- Flexibility to mix SSR, CSR & static generation

UI Library

Tailwind CSS for styling. Additionally **ShadCN** could be used.

The platform needs a clean, user-friendly, and fast UI that works well across all devices. Tailwind CSS and ShadCN provide the best combination of flexibility, performance, and design consistency for this project. Tailwind CSS keeps everything organized with utility classes. No extra CSS files, easy to maintain & update styles quickly, scalable for future growth. Since many users will browse the website on mobile devices, we need a UI that automatically adapts to different screen sizes. Tailwind makes responsiveness super easy with mobile-first classes.

ShadCN provides modern UI components (buttons, forms, modals, dropdowns) and gives you beautiful pre-built components, but we still need custom styles for branding. Tailwind allows you to customize ShadCN components without writing extra CSS.

Hosting

Vercel for serverless deployments and scalability. Since the platform needs to be fast, scalable, and easy to deploy, Vercel is the best choice because it offers:

- Seamless Next.js Integration (Zero-config deployment)
- Automatic Global CDN (Super-fast load times across Europe)
- Serverless Functions (No need to manage backend servers)
- Auto-Scaling (Handles traffic spikes smoothly)
- Built-in CI/CD (Instant updates with every code push)
- Free Tier & Cost Efficiency (Great for startups)

Seamless Next.js Integration (Zero-Config Deployment). Vercel is built for Next.js, making deployment as easy as:

1. Connect your GitHub repository
2. Click “Deploy”
3. Done! Your website is live.

No need to configure servers, domains, or manual builds

AI Tools for Frontend development

- **Uizard** – AI-powered UI design tool that converts sketches into functional prototypes.
- **ChatGPT (especially with GPT-4 Turbo)**: Deep understanding of frontend and backend technologies (React, Next.js, Tailwind CSS, SQL, API integration), generate reusable React components and hooks, debug, full-stack guidance (frontend + backend + API handling), step-by-step debugging & explanations, complex UI interactions and real-world implementations (e.g., dynamic search filters), teaching best practices in web development.
- **Figma (AI Plugins)** – AI-powered design suggestions and auto-layout assistance, prototype user interfaces.
- **Canva (AI Tools)** – AI-generated design elements and website mockups.
- **Framer AI** – AI-powered website builder that creates code from design.
- **Locofy AI** – Converts Figma and Adobe XD designs into front-end code (React, HTML, CSS).
- **DivMagic** – AI-powered tool that extracts front-end code from UI components.
- **Durable AI** – AI-powered website builder that creates fully responsive pages.
- **DALL-E** – for graphic design work. For ex designing the logo and other tasks which normally requires a graphic designer, creating unique dog-related icons, banners, or illustrations.
- **Google Gemini** (formerly Bard) – Free & good for coding/design questions.

Core Frontend Features and Pages:

1. Home Page

- **Main Start Page:** Introductory section with a welcoming message and an engaging visual (possibly your mascot dog picture).

- **Navigation Menu:** Links to key sections: Home, Our Mission, Donate, Useful Info & Links, Happy Stories.
- **Dog Search Section:** A primary feature allowing users to filter dogs based on:
    - **Sex:** All, Female, Male
    - **Age:** 0m-1yr, 1-5yrs, 6+yrs
    - **Size:** Small, Medium, Large, X-Large
    - **Country:** All EU, Search by country
    - **Breed:** All, Search by breed
    - **ID Number:** Search by unique ID
- **Motivational & Cute Dog Photos:** To inspire users to adopt.

2. Our Mission Page

- **Story Behind the Website:** Explain why the platform was created.
- **Goals & Impact:** What changes the platform aims to bring.
- **Why This Website is Needed:** Information on the shelter dog adoption crisis and how users can help.

---

3. Donate & Donation Shop Page

- **Donate to Our Mission:** Explain how donations help.
- **Donation Link:** Direct link to process donations.
- **List of Donation Goals:** Show different use cases (e.g., medical expenses, food, transport).
- **Donation Shop:** A section to sell merch/items supporting shelters.

---

4. Useful Info & Links Page

- **Transport Within EU:** Guide on pet transport regulations.
- **Paperwork Guide:** Documentation required for adopting a shelter dog.
- **Adoption Tips:** What to keep in mind before adopting.
- **Why Shelter Dogs Are a Great Choice:** Benefits of adopting from shelters.
- **Health & Training Advice:** General recommendations and useful links.

---

5. Happy Stories & Blog Page

- **Success Stories:** Showcases of successful adoptions.
- **Stories from Website Adoptions:** Future section for dogs adopted through your platform.
- **Community Blog:** Users can post photos/videos of their adopted dogs.

---

6. Multi-Language Support

- **Language Menu:** Allow users to select their preferred language.

Multilingual Support

See the "*Translation of webpage and real time messages*" document and choose between the first recommendation, the easiest way of how to do it and the most efficient way using best practices. Discuss together with the back end developers.

API development process

See the *"API development process"* document for an overview and step by step guidance.

API integrations

API integrations = dynamic, real-time content.

Without APIs, we need static pages that never update.

With APIs, the platform will be interactive, fast, and always up-to-date**.**

Implement API integrations for dynamic content. Frontend (Next.js/React) will connect to an API (backend or third-party services) to fetch, display, and update live data.

Frontend will need real-time data from a backend database or third-party services. Here are the main areas where API integration is essential:

Dog Search & Profiles

API Needed: Fetch a list of adoptable dogs from the database.

- Users select filters (breed, country, size, etc.).
- Frontend calls the API to get matching dogs.
- Results update without reloading the page.

Shelter Dashboard (For Managing Listed Dogs)

API Needed: Allow shelters to add, update, or remove dogs.

- Shelters manage their dog listings.
- Changes instantly reflect on the frontend.

Donations & Payment Integration

API Needed: Connect to a payment gateway (e.g., Stripe, PayPal).

- User clicks Donate → API request to generate a payment link.
- Payment is processed → API sends confirmation.
- Frontend updates to thank the donor.

Happy Stories & Blog

API Needed**:** Users upload stories & images.

- Success stories update automatically when a new one is posted.
- API fetches latest blog posts without a page refresh.

Multi-Language Support

API Needed: Fetch translations dynamically.
Example: If a user selects German, the frontend calls:

```
fetch('/api/translate?lang=de')
```

Text updates to German dynamically.

Use Google Maps API for location-based search results

Using Google Maps API, the adoption platform will:

- Let users search by location (city, country, or nearby shelters).
- Show shelters on a map with clickable markers.
- Auto-detect user location for a better experience.

Most important to get the “search by country” function to work.

How to implement API integration in Next.js

- Use Next.js API Routes (`/pages/api/`) to handle backend requests.
- Use fetch() or Axios in React components to call APIs.
- Use SWR (Stale-While-Revalidate) for automatic data refreshing.
- Use Zustand or React Context to manage global state (like user logins).

Form Handling & Search: Implement an advanced search system with filters

For the European Shelter Dog Adoption Platform, handling forms and implementing an advanced search system with filters is crucial for helping users find dogs that match their preferences. Here’s how it applies to the project:

Form Handling (User Input for Search)

We need a form where users can enter their search criteria, such as:

- Sex
- Age
- Size
- Country
- Breed
- ID number

I will build this using React with ShadCN (for UI components) and handle the form submission with Next.js API routes or client-side state management.

Advanced Search with Filters (Efficient Data Retrieval)

Once the user submits the form, the search system should:

1. Send the search query to the backend (via an API route in Next.js).
2. Retrieve filtered data from our database (e.g., using SQL queries with WHERE conditions).
3. Display search results dynamically in a responsive UI.

Tech Stack & Implementation

Frontend:

- Use Next.js + React + Tailwind CSS + ShadCN to build the search form.
- Use React Hook Form or ShadCN form components for better validation and submission handling.

Backend:

- Create an API route (`/api/search`) in Next.js to process search queries.
- Use SQL queries (e.g., `SELECT * FROM dogs WHERE breed = 'Labrador' AND location = 'Germany'`).
- Optimize queries with indexes and pagination.

Responsive Design: Mobile-friendly layout

Since users will access our website on various devices, making it mobile-friendly is crucial. Here's how to approach responsive design in my frontend development process:

Plan Mobile-First Design

Start designing for small screens first, then scale up for larger screens.

- Prioritize essential elements (Search, Filters, Dog profiles, Upper menu).
- Ensure key actions are easily accessible.

Example:
Our search page should display results in a single-column layout on mobile and a grid layout on larger screens.

Use Tailwind CSS for Responsive Layouts

Tailwind provides built-in responsive utilities, making it easy to adapt layouts for different screen sizes. Example: responsive grid for dog listings:

grid-cols-1: Single column on small screens.

sm:grid-cols-2: Two columns on small devices.

md:grid-cols-3: Three columns on tablets.

lg:grid-cols-4: Four columns on desktops.

Optimize Navigation for Mobile

Use a hamburger menu on smaller screens instead of a full navigation bar.

Example: Responsive Navbar with ShadCN:

hidden md:flex: The navigation links are hidden on mobile and only show on md (medium screens) and larger.

md:hidden: The hamburger menu only appears on small screens.

Make Forms & Buttons Mobile-Friendly

- Use larger tap targets (buttons at least 44x44px).
- Ensure inputs are easy to tap.
- Optimize for touch keyboards (use `type="email"` for email fields).

Example: Mobile-Optimized Search Form:

flex flex-col sm:flex-row: Stacks inputs vertically on mobile, but in a row on larger screens.

w-full sm:w-auto: The input takes full width on mobile but shrinks on larger screens.

Test on Different Devices

- Use Chrome DevTools (F12 → Toggle Device Toolbar).
- Test on real phones & tablets.
- Check for touch-friendliness, font size, and tap areas.

For an **MVP (Minimum Viable Product)**, basic responsiveness is necessary, but I don't need to perfect every detail. Focus on:

- Ensuring key features work on mobile (search, dog profiles, navigation)
- Using Tailwind's responsive classes for quick mobile adaptability.
- Avoiding broken layouts (no overflowing content or unreadable text).
- Skipping pixel-perfect styling—you can refine the UI later.

MVP-Level Responsive Priorities

1. Single-column layout on mobile (e.g., one dog per row in search results).
2. Simple hamburger menu for navigation (or just a basic top menu).
3. Forms that are easy to use on touchscreens (inputs not too small).
4. Basic testing on mobile devices to catch major issues.

What Can Wait?

- Fancy animations or transitions.
- Perfecting grid layouts for all screen sizes.
- Advanced UI components (modals, dropdowns with animations, etc.).

For the MVP, just make sure the core UX is functional on mobile.

Study plan for being able to create MVP

Learning the basics of **JavaScript, HTML, CSS and React** is essential before diving into **Next.js**.

Learning path for mandatory knowledge needed for creating MVP

1. **HTML basics** structure of web pages (1 day)
2. **CSS & Tailwind CSS basics** styling & layout (2 + 1 day)
3. **JavaScript fundamentals** Core programming concepts (variables, functions, loops, events, DOM manipulation) (1 week)
4. **ES6+ Features** – Arrow functions, destructuring, promises, async/await. These modern JavaScript features are heavily used in **React, Next.js, and APIs**, making your code cleaner and easier to manage.
5. **React basics** (1 week)
6. **Next.js fundamentals** (1 week)
7. **Git & GitHub basics** (2 days)

Total study time: 3.5 weeks

Not mandatory for creating MVP but useful for further development

1. **TypeScript** – Adds type safety but is not strictly necessary for an MVP. You can start with JavaScript.
2. **ShadCN** – Optional but useful for pre-styled components.
3. **Basic API handling** – Fetching shelter dog data from the database (e.g., using `fetch()` or Next.js API routes).

Resources to Learn:

I will use a combination of Chat GPT and Youtube videos for a lot of the learning.

1. HTML & CSS & Tailwind CSS

- MDN Web Docs – Great for learning HTML, CSS, and JavaScript
- FreeCodeCamp / The Odin Project – Step-by-step courses
- CSS Grid & Flexbox Cheatsheets: CSS Tricks

- Tailwind CSS Docs: tailwindcss.com

2. JavaScript & ES6+

- JavaScript.info: JavaScript Guide
- Eloquent JavaScript (book): Read Online
- JavaScript30 (Wes Bos) – Hands-on projects: JavaScript30

3. React.js

- Official React Docs: react.dev
- Scrimba's Interactive React Course: React Course
- Full React Playlist by The Net Ninja: YouTube
- "React for Beginners" by Wes Bos (paid)

4. Next.js

- Official Next.js Docs: nextjs.org
- Full Next.js Course (Traversy Media): YouTube

5. Git & GitHub Basics

- Youtube: The Net Ninja: Git & GitHub for Beginners
  Short and concise videos explaining Git concepts and GitHub workflows.
- Youtube: Traversy Media: Git and GitHub Crash Course. A 1-hour beginner-friendly crash course covering Git basics and GitHub essentials.

6. ShadCN

- ShadCN Docs: shadcn/ui

Learning guidance (what to learn and in which order)

1. **HTML**

HTML is the backbone of web development. Everything else builds on it.
What to focus on?

- Basic structure (`<html>`, `<head>`, `<body>`).
- Common elements (`<div>`, `<h1>`, `<p>`, `<a>`, `<img>`).

- Semantic tags (`<header>`, `<footer>`, `<main>`, `<section>`).
- Forms and attributes.

2. **CSS**

CSS is essential for styling and layout. Master it before diving into frameworks like Tailwind CSS.

What to focus on?

- CSS basics: syntax, selectors, and properties.
- Box model, Flexbox, and Grid for layout.
- Responsive design with media queries.
- Pseudo-classes (`:hover`, `:nth-child()`) and pseudo-elements (`::before`, `::after`).

3. **Tailwind CSS**

What to focus on?

- Setting up Tailwind CSS in projects.
- Using utility classes for spacing, colors, typography, and layout.
- Responsive design with Tailwind.
- Customizing Tailwind with `tailwind.config.js`.

4. **JavaScript (ES6+)**

JavaScript brings interactivity and logic to your web projects.

What to focus on?

- Basics: variables (`let`, `const`), data types, functions, loops, conditionals.
- DOM manipulation and events: selecting elements, event listeners, modifying content/styles.
- Fetch API and basic asynchronous programming.
- Practice problem-solving with small exercises

- **ES6+ features:** 1. Arrow functions (`() => {}`), template literals (``), and destructuring 2. Modules (`import/export`) and `async/await`.
- Writing cleaner, modern code.
- Understanding how `async/await` works with API calls

5. **<u>React</u>**

React is a JavaScript library for building user interfaces, so understanding JavaScript is crucial first. Next.js is based on React, so understanding React is crucial.

What to focus on?

- JSX syntax.
- Components (functional components).
- Props and state management.
- Event handling, conditional rendering, lifecycle basics
- React hooks (`useState`, `useEffect`, etc.).

6. **<u>Next js</u>**

Once comfortable with React, start with Next.js, as it will be my primary framework for the project. Next.js builds on React by adding server-side rendering, routing, and other advanced features.

What to focus on?

- File-based routing and dynamic routing.
- Data fetching: `getStaticProps`, `getServerSideProps`, and `getStaticPaths`. Static site generation (SSG) and server-side rendering (SSR)
- API routes.
- Deploying Next.js applications (e.g., Vercel).

7. **<u>Git & GitHub Basics</u>**

Git basics (commits, branches, merging) and GitHub workflows (push, pull, creating repositories). Hands-on practice.

Suggested Study Plan for 2 Days

Day 1: Learn Git Basics

- Understand version control concepts.
- Learn key Git commands: `git init`, `git add`, `git commit`, `git status`, `git log`.
- Create a local repository and practice basic operations.

Day 2: Learn GitHub Basics

- Create a GitHub account.
- Practice pushing a local repo to GitHub (`git remote add`, `git push`).
- Understand cloning and forking repositories.
- Learn about pull requests and GitHub collaboration workflows.

8. **ShadCN**

ShadCN combines React and utility classes (like Tailwind CSS) to create pre-built UI components. You need a strong foundation in React and Tailwind to use it effectively.

What to focus on?

- Setting up ShadCN components in a Next.js project.
- Customizing and extending pre-built components.
- Creating accessible and reusable design systems.

Recommendation for Practicing MVP Development

1. **HTML + CSS + JavaScript (2 Weeks Total)**
    - Build a static webpage with dynamic JavaScript elements like form validation, event listeners, or simple animations.
2. **React + Tailwind CSS (1 Week)**
    - Recreate the webpage as a React app, styling it with Tailwind CSS. Add interactivity with state and hooks.
3. **Next.js + Git (1 Week)**

- Convert the React app to a Next.js project. Practice routing, server-side rendering, and deploying on Vercel.

Start the frontend development

Now that I have the necessary skills, I will begin working on the shelter dog adoption platform frontend:

1. **Set up a Next.js project**
2. **Create pages** – Home, Search, Dog Profile, Shelter Dashboard
3. **Integrate API calls** to fetch dog and shelter data
4. **Implement UI components using ShadCN & Tailwind CSS**
5. **Deploy on Vercel**

Detailed step by step front end development plan

I will start creating this detailed step by step plan and documentation when I start the development process.

Further development suggestions

Chat GPT: The frontend overview you provided is comprehensive and covers the critical components of building the platform, including the framework, UI library, hosting, tools, core features, multilingual support, API integration, responsive design, and a detailed learning path for developers. However, there are a few areas where additional clarification or expansion could enhance the document's utility:

Suggestions for Improvement:

1. Accessibility:
    - Ensure the platform is inclusive by incorporating accessibility standards (e.g., WCAG compliance).
    - Mention the need for features like alt text for images, keyboard navigability, and ARIA roles.

2. Error Handling:
    - Include strategies for handling errors in API responses and user inputs to improve user experience.
    - Consider specifying user feedback for failed form submissions or unsuccessful searches.
3. Testing:
    - Outline the importance of frontend testing frameworks like Jest, Cypress, or React Testing Library for unit, integration, and end-to-end tests.
4. Analytics and Monitoring:
    - Add a section on integrating analytics (e.g., Google Analytics) for tracking user behavior.
    - Mention monitoring tools like Sentry for error tracking in the deployed application.
5. Scalability for Future Features:
    - Highlight plans for modular design to easily integrate future features, like additional search filters or new content sections.
6. User Onboarding/Guides:
    - Consider a basic user guide or onboarding process for first-time visitors to navigate features like advanced search and donation tools.
7. Performance Optimization:
    - Though you've mentioned performance aspects (e.g., Vercel, Next.js SSR), expanding on techniques like lazy loading, caching strategies, and code splitting can be valuable.
8. Design Feedback Loops:
    - Recommend periodic usability testing and surveys to refine UI/UX based on user feedback.

Including these points would strengthen the document's readiness to address all facets of frontend development and deployment effectively. Let me know if you'd like help drafting additional content for these suggestions!